

PHASE 2 DOCUMENT

EduGenie: Google Gemini Powered Learning Assistant

Phase Name: System Design and Architecture

Project Name	EduGenie: Google Gemini Powered Learning Assistant
Phase	Phase 2 – System Design and Architecture
Project Type	AI-Powered Learning Application
Main Technology	Google Gemini API
Prepared By	_____
Guide / Faculty	_____
Date	_____

Academic Project Phase Documentation

1. Phase Overview

This is Phase 2 of **EduGenie: Google Gemini Powered Learning Assistant**. Based on the requirements identified in Phase 1, this phase defines the technical structure of the application. It describes the proposed architecture, core modules, data flow, user-interface flow, database entities, user roles, and secure integration boundary for Google Gemini.

2. Design Goals

The system design should make EduGenie easy for students to use, secure for user data and API credentials, modular for future expansion, responsive across devices, and maintainable for developers. The design separates the user interface from backend logic so that the Google Gemini API key remains protected and application features can be updated without changing the entire system.

3. High-Level Architecture

EduGenie follows a layered client-server architecture. The frontend provides student, faculty, and administrator screens. The backend manages authentication, learning requests, validation, prompt construction, Gemini API communication, and database operations. The database stores authorized user profiles, saved learning content, quiz history, and application configuration. Google Gemini is accessed only from the backend through a secure API integration layer.

4. Architecture Components

Presentation Layer: Web pages or mobile-responsive screens for login, dashboard, AI chat, quiz, study planner, project assistant, and profile settings.

Application Layer: Backend services for business rules, user sessions, feature control, request validation, prompt management, response formatting, and error handling.

Data Layer: Database services for users, roles, learning sessions, saved content, quiz attempts, study plans, and audit records.

AI Integration Layer: A protected backend module that sends approved prompts to Google Gemini and returns validated results to EduGenie.

Security Layer: Authentication, authorization, access control, secure secret storage, input validation, logging, and rate control.

5. Major Modules

1. Authentication and Profile Module: Handles registration, login, logout, password protection, and profile preferences.
2. AI Learning Chat Module: Accepts student questions and displays Gemini-powered explanations.
3. Quiz Generator Module: Creates practice questions, stores attempts, and displays results.
4. Study Planner Module: Produces revision schedules based on selected subjects, goals, and

available time.

5. Notes and Summary Module: Helps users save responses, generate summaries, and revisit learning content.

6. Project Assistant Module: Supports project titles, objectives, module ideas, documentation outlines, and presentation preparation.

7. Faculty Module: Provides optional resource sharing, approved prompts, and class-level insights.

8. Admin Module: Manages users, roles, categories, configurations, and policy settings.

6. User Roles and Access

Students can access their own learning sessions, ask questions, generate quizzes, create study plans, and save content. Faculty members can access faculty-enabled resources and optionally share learning guidance with students. Administrators can manage users, roles, subject categories, system settings, and access policies. Role-based access control ensures that each user receives only the permissions required for their role.

7. User Interface Design

The proposed interface begins with a secure login screen. After login, the student dashboard displays quick actions such as Ask EduGenie, Generate Quiz, Create Study Plan, Summarize Notes, and Project Help. The chat screen includes a prompt input area, learning-mode selector, answer display, save option, and follow-up question support. The design uses clear navigation, readable text, simple actions, helpful loading states, and mobile-responsive layouts.

8. Data Design

The proposed database includes a Users entity for account details and roles; LearningSessions for chat interactions; SavedContent for notes, summaries, and saved responses; QuizAttempts for generated quizzes and answers; StudyPlans for schedules and tasks; Projects for optional project-guidance records; Resources for faculty-shared learning material; and AuditLogs for administrative events. Sensitive fields should be minimized, access-controlled, and retained only according to the required policy.

9. Data Flow

1. The user logs in through the frontend.
2. The frontend sends authenticated requests to the backend.
3. The backend checks the user role and validates the request.
4. For AI features, the backend creates a structured prompt based on the selected learning mode and user input.
5. The backend sends the request to Google Gemini.
6. Gemini returns generated content to the backend.
7. The backend performs formatting, safety checks, and error handling.
8. The frontend displays the result to the user.
9. If enabled and authorized, relevant learning data is stored in the database.

10. Secure Gemini Integration Design

The Google Gemini API key will be kept only in backend environment variables or a secrets-management service. The key must not be embedded in frontend source code, browser storage, screenshots, or public repositories. The backend will expose an internal API endpoint such as an AI request service. This service validates user access, limits unnecessary requests, constructs a clear prompt, sends it to Gemini, handles service errors, and returns an appropriate response to the logged-in user.

11. Error Handling and Safety

The design includes user-friendly messages for network failure, invalid input, expired sessions, unavailable AI service, and unexpected server errors. Inputs should be validated before requests are sent. The system should avoid presenting AI output as guaranteed fact and may show reminders to verify academic content. Logs should support troubleshooting while avoiding storage of unnecessary sensitive information.

12. Technology Design

The frontend can be implemented using React, Angular, or standard HTML/CSS/JavaScript. The backend can use Node.js with Express or Python with FastAPI/Django. A database such as Firebase, PostgreSQL, or MongoDB can store application data. Google Gemini provides the generative AI capability. Deployment can use an approved cloud service. The final selection should consider developer skills, institutional infrastructure, cost, and security requirements.

13. Phase Deliverables

The deliverables of Phase 2 are the system architecture definition, identified application modules, user-role design, UI navigation flow, database entity plan, data-flow description, Google Gemini integration design, security considerations, and an implementation roadmap for the development phase.

14. Planned Next Phase

The next phase will be **Phase 3: Google Gemini API Integration and Core Backend Development**. It will implement the secure backend connection, API configuration, prompt-processing service, request validation, response handling, and initial AI learning-assistant functionality.

15. Conclusion

Phase 2 converts the Phase 1 requirements into a structured technical design for EduGenie. The proposed architecture keeps the user experience simple while protecting user data and the Gemini API key through a backend integration layer. This design provides a scalable base for implementing the AI-powered learning assistant in the next phase.

Phase Approval

Prepared by	_____
-------------	-------

Guide / Faculty	
Approved by	